

重大 AI 更新提醒

06-30 | 智能体框架、RL 训练栈与推理修复

本期导读

本期聚焦 Google ADK Go 2.0 发布、PyTorch 原生 RL 训练框架 Miles 开源，以及 llama.cpp 多项 CUDA 修复。

已检查来源

Google Developers Blog、GitHub Release、PyTorch Blog

1. Google 发布 Agent Development Kit (ADK) for Go 2.0：图 workflow 引擎与人工介入编排

来源 Google Developers Blog | 类型 官方发布 主题 智能体与开发者平台 时间 06-30 14:46 | 分数 7

是什么	Google 发布了 Agent Development Kit (ADK) for Go 2.0，这是一个用于构建多智能体应用的开发工具包。新版本引入了基于图的流程引擎，支持人工在环（HITL）编排、动态执行和自动弹性恢复（如指数退避重试）。
通俗解释	想象你要搭建一个由多个 AI 助手组成的团队，每个助手负责不同任务（比如一个查天气、一个订酒店）。ADK Go 2.0 提供了一个可视化 流程图 工具，让你能像画流程图一样定义这些助手如何协作。它还允许在关键步骤插入人工审核（比如订酒店前让用户确认），并且如果某个助手出错，系统会自动重试。所有助手共享同一个运行环境，方便监控和保存状态。
主要作用	解决多智能体应用开发中流程编排复杂、人工介入困难、系统稳定性不足的问题。对于需要构建可靠多智能体系统的开发者来说，这是一个重要的基础设施升级。
核心功能	基于图的流程引擎：支持复杂多智能体工作流定义；内置人工在环（HITL）原语：可在流程中插入人工审批或干预；动态执行：使用纯 Go 代码实现灵活的执行逻辑；自动弹性恢复：指数退避重试等机制提升系统稳定性
对比判断	暂无可信公开对比数据
适合谁	适合使用 Go 语言开发多智能体应用的工程师、AI 平台团队，以及需要构建复杂自动化工作流的企业开发者。

直达链接 <https://developers.googleblog.com/announcing-adk-go-20/>

2. Miles : PyTorch 原生的大规模 LLM 强化学习后训练框架

来源 PyTorch Blog | 类型 官方发布 | 主题 多模态理解 | 时间 06-30 12:48 | 分数 7

是什么	Miles 是 RadixArk 开源的 PyTorch 原生框架，用于大规模 LLM 的强化学习后训练。它整合了 SGLang 进行推理、NVIDIA Megatron-LM 进行训练、Ray 进行任务编排，并通过 PyTorch 原生扩展性提供可插拔接口。
通俗解释	训练一个大语言模型（比如 ChatGPT）通常分两步：先预训练（学知识），再后训练（学对话）。后训练常用强化学习（RL），但 RL 训练需要同时运行推理和训练，非常复杂。Miles 就像一个 组装套件，把 SGLang（高效推理）、Megatron-LM（大规模训练）、Ray（分布式调度）这些成熟组件拼在一起，而且全部基于 PyTorch，方便研究者自定义。比如你想让模型学会写诗，只需插入一个奖励函数，Miles 就能自动协调推理和训练流程。
主要作用	降低大规模 LLM 强化学习后训练的门槛，提供一套可扩展、高性能的 PyTorch 原生解决方案，让研究者和工程师能更轻松地实验 RL 算法。
核心功能	PyTorch 原生：完全兼容 PyTorch 生态，易于扩展；组件化集成：组合 SGLang、Megatron-LM、Ray 等成熟工具；可插拔接口：支持自定义奖励函数、策略等；大规模支持：面向数百 GPU 级别的训练场景
对比判断	暂无可信公开对比数据
适合谁	适合从事 LLM 后训练（RLHF/RL）的研究人员、AI 训练工程师，以及需要大规模 RL 训练平台的团队。
直达链接	https://pytorch.org/blog/miles-a-pytorch-native-stack-for-large-scale-llm-rl-post-training/

3. llama.cpp b9851 : 修复 CUDA 下 KQ mask strides 整数截断与溢出问题

来源 GitHub Release | 类型 GitHub Release | 主题 推理性能与基础设施 | 时间 06-30 14:35 | 分数 7

是什么	llama.cpp 发布 b9851 版本，修复了在 CUDA 内核 flash_attn_mask_to_KV_max 中，使用 KQ mask strides 时可能出现的整数截断和溢出错误。
通俗解释	llama.cpp 是一个在本地运行大语言模型的工具。这次修复针对的是 GPU 上计算注意力机制时的一个 bug。注意力机制中有一个 掩码（mask）用来告诉模型哪些位置需要忽略，而 strides 是数据在内存中的步长。当掩码的步长很大时，整数计算可能溢出，导致错误结果。这个修复就像给计算器加了一个 防溢出 功能，确保大数值也能正确计算。
主要作用	修复一个可能导致模型输出错误或崩溃的 CUDA 内核 bug，提升 llama.cpp 在特定场景下的稳定性和正确性。
核心功能	修复整数截断：防止大 strides 被错误截断；修复溢出错误：避免整数溢出导致的计算异常；仅影响 CUDA 后端：不影响 CPU 或其他后端
对比判断	暂无可信公开对比数据
适合谁	适合使用 llama.cpp 在 NVIDIA GPU 上运行大语言模型的开发者，尤其是使用 KQ mask 的注意力变体（如某些长上下文模型）的用户。
直达链接	https://github.com/ggml-org/llama.cpp/releases/tag/b9851

以上信息基于候选源整理，请以官方发布为准。